

Sistemas Compostos de IA: Otimização de Respostas em Benchmarks Objetivos por meio de Orquestração Baseada no Ralph Loop

Hugo Emílio Nomura, Pedro Henrique Correia de Oliveira, Pedro Henrique Satoru Lima Takahashi, Vitor Monteiro Vianna

Ciência da Computação – Centro Universitário FEI, campus São Paulo

unifhnomura@fei.edu.br, unifpoliveira@fei.edu.br, unifptakahashi@fei.edu.br, unievvianna@fei.edu.br

Orientador: Charles Henrique Porto Ferreira

Departamento de Ciência da Computação, Centro Universitário FEI

cferreira@fei.edu.br

Resumo: Embora os *Large Language Models* (LLMs) continuem operando como geradores de texto, sua evolução recente não depende apenas do aumento de capacidade ou da complexidade matemática. Soluções modernas incorporam planejamento, refinamento e ciclos iterativos de raciocínio para melhorar a qualidade e a consistência das saídas. No entanto, o uso prolongado desses mecanismos amplia a janela de contexto e pode causar degradação de informação (*context rot*), reduzindo a confiabilidade das respostas.

Este trabalho propõe adaptar os princípios do Ralph Wiggum Loop para cenários gerais de *benchmarking*, com o objetivo de melhorar respostas por meio de chamadas iterativas ao próprio modelo, sem ferramentas externas. Para mitigar o *context rot*, a arquitetura preserva um comportamento *stateless* entre as fases, mantendo apenas os artefatos estritamente necessários a cada etapa.

O impacto empírico da abordagem será avaliado com os *benchmarks* MMLU e GPQA, comparando a acurácia de chamadas diretas ao modelo com a de execuções mediadas pelo *harness* proposto.

Palavras-chaves: *Large Language Models* (LLMs). Ralph Wiggum Loop. Sistemas Compostos de IA. Engenharia de Prompt. Decaimento de Contexto (*context rot*).

I. Introdução

Os *Large Language Models* (LLMs) consolidaram-se como o núcleo de uma nova geração de sistemas capazes de executar tarefas complexas de processamento de linguagem natural, incluindo geração de texto, resolução de problemas, planejamento intermediário e suporte à tomada de decisão. Apesar dessas capacidades, os modelos que servem como base para esses sistemas continuam sendo, em essência, geradores probabilísticos de *tokens*, operando a partir de uma entrada de texto e retornando como saída outro texto. Nesse contexto, parte significativa da evolução recente dos sistemas baseados em LLMs passou a ocorrer não apenas nos próprios modelos, mas também nas estruturas responsáveis por organizar sua utilização prática. Trabalhos recentes, como [1], afirmam que “*The performance of large language model (LLM) systems depends not only on model weights, but also on their harness: the code that determines what information to store, retrieve, and present to the model*”, evidenciando o papel dos chamados *harnesses* na ampliação das capacidades desses sistemas. Ferramentas modernas, como o *ChatGPT*, incorporam tais mecanismos para adicionar camadas de coordenação, decomposição de tarefas, validação intermediária e orquestração de múltiplas inferências, permitindo que o modelo execute fluxos significativamente mais sofisticados do que uma única chamada linear de entrada e saída.

Abordagens baseadas em uma única chamada ao modelo, nas quais toda a resposta é produzida em uma única

etapa de inferência, apresentam limitações importantes em tarefas que exigem raciocínio complexo, múltiplas etapas de decisão ou validações intermediárias. Como resposta a esse problema, diferentes linhas de pesquisa passaram a explorar fluxos estruturados de *thinking*, nos quais o processo de inferência é explicitamente decomposto em etapas sucessivas de raciocínio, planejamento, execução e refinamento. Trabalhos baseados em *Chain-of-Thought* [2], *Tree of Thoughts* [3] e na técnica de *Least-to-Most* [4] demonstram que a decomposição explícita do raciocínio favorece significativamente a resolução de tarefas complexas. De maneira semelhante, abordagens como *ReAct* [5] combinam raciocínio e ações iterativas, enquanto métodos como *Reflexion* [6], *Self-Debugging* [7] e *Self-Refine* [8] exploram ciclos sucessivos de avaliação, crítica e refinamento da própria resposta gerada. Em conjunto, esses trabalhos indicam que ganhos de desempenho não dependem exclusivamente do tamanho do modelo ou da quantidade de *tokens* gerados, mas também da maneira como o processo de inferência é estruturado, coordenado e validado ao longo da execução.

Apesar dos ganhos observados nessas arquiteturas iterativas compostas por agentes, ciclos de reflexão ou *pipelines* sucessivos, tais abordagens também introduzem novos desafios relacionados ao gerenciamento do contexto acumulado durante a inferência. Em muitos desses sistemas, o histórico completo de interações permanece armazenado na janela de contexto ao longo de toda a execução como mecanismo de preservação da continuidade do raciocínio.

Embora essa estratégia favoreça rastreabilidade e coerência entre etapas intermediárias, ela também aumenta progressivamente a dificuldade de recuperação de informações relevantes conforme o contexto cresce. Liu et al. [9], por exemplo, demonstram que ocorre perda de desempenho em tarefas que exigem localizar informações relevantes em sequências extensas. Trabalhos recentes também passaram a discutir fenômenos associados à degradação progressiva do contexto em execuções longas, incluindo compressão implícita de informações, perda de relevância contextual e aumento de inconsistências ao longo da inferência. Esses efeitos vêm sendo frequentemente associados, ao conceito de *context rot*, termo utilizado por Hong et al. [10] para descrever a degradação gradual da qualidade das respostas conforme o volume de contexto acumulado aumenta.

Além do avanço acadêmico dessas abordagens, a indústria de engenharia de software também passou a investir intensamente em arquiteturas de *deep thinking* e em *harnesses* especializados para ampliar a capacidade operacional de LLMs em tarefas complexas de desenvolvimento. Ferramentas recentes, como *Claude Code*, *GPT Codex*, *OpenCLI* e *Devin*, exemplificam essa tendência ao combinar planejamento, decomposição de tarefas, validação iterativa e execução coordenada de múltiplas inferências. Nesse contexto, uma das abordagens que ganhou notoriedade foi o *Ralph Wiggum Loop* [11], popularizado por Geoffrey Huntley no contexto de engenharia de software. Em sua formulação original, entretanto, o modelo não foi concebido para cenários generalistas, mas especificamente para fluxos de desenvolvimento baseados em conceitos próprios da engenharia de software, como definição de funcionalidades, requisitos, critérios de aceite e integração com sistemas de versionamento, como o *Git*. Ainda assim, sua estrutura apresenta dois princípios centrais relevantes para os problemas discutidos na literatura de LLMs: a decomposição do processo em tarefas independentes, reduzindo a necessidade de reaproveitamento contínuo do contexto entre etapas, e a execução iterativa de cada tarefa até que critérios mínimos de qualidade sejam satisfatoriamente atingidos.

Este trabalho parte de três premissas consolidadas na literatura: que abordagens de *thinking* podem melhorar a qualidade das respostas de LLMs em cenários específicos, como os de tarefas complexas; que tais abordagens introduzem problemas associados ao crescimento excessivo do contexto acumulado, fenômeno descrito como *context rot*; e que o *Ralph Wiggum Loop* [11], concebido por Huntley para fluxos de engenharia de software, oferece princípios estruturais capazes de mitigar o problema de *context rot* por meio de decomposição de tarefas e isolamento contextual entre etapas. A partir dessas premissas, propõe-se a adaptação do *Ralph Wiggum Loop* para um contexto generalista de resolução de tarefas em linguagem natural a partir de um *prompt* de entrada fornecido pelo usuário. Diferentemente da proposta original, centrada em desenvolvimento de software, definições de funcionalidades, critérios de aceite baseados em requisitos funcionais e integração com ferramentas de versionamento (como *Git*), a adaptação proposta investiga se, ao adaptar a estrutura do *Ralph Wiggum*

Loop a um fluxo completo de *thinking* aplicável à resolução de perguntas gerais e à exploração de problemas diversos e de diferentes complexidades, também haverá melhora na qualidade dessas respostas quando comparadas a uma única etapa de inferência.

Para avaliar essa hipótese de maneira objetiva, reproduzível e quantitativa, o trabalho compara diretamente duas estratégias de inferência: (i) uma chamada tradicional a um modelo de LLM, na qual toda a resposta é produzida em uma única etapa de inferência; e (ii) uma execução mediada pelo *harness* proposto, que adapta o *Ralph Wiggum Loop* para cenários generalistas. O objetivo experimental é medir em que magnitude a aplicação desse fluxo estruturado de *thinking* melhora a qualidade das respostas em relação à inferência singular tradicional. Para permitir essa quantificação, os experimentos utilizam *benchmarks* compostos por questões e respostas previamente definidas, estratégia amplamente adotada para mensuração de ganhos em arquiteturas de raciocínio e orquestração de LLMs, e usada em trabalhos como *ReAct* [5] e *Beyond the Strongest LLM* [12]. Os experimentos são conduzidos com diferentes modelos da família Llama 3.1, com diferentes quantidades de parâmetros utilizados na execução do modelo, representando distintos níveis de complexidade computacional, e avaliados por meio dos *benchmarks* MMLU [13] e GPQA [14], sendo a acurácia, medida pela quantidade de acertos entre a alternativa selecionada pelo sistema de LLM e o gabarito oficial.

II. Referencial Bibliografico

Esta seção apresenta os conceitos fundamentais necessários para a compreensão do trabalho proposto. São definidos os termos técnicos centrais utilizados ao longo do texto, descritas as principais estratégias de raciocínio e refinamento aplicadas a modelos de linguagem, e introduzido o paradigma de orquestração adotado neste trabalho.

A. Modelos de Linguagem e a Arquitetura de Harness

Embora os *Large Language Models* (LLMs) [15] sejam amplamente consolidados como geradores probabilísticos de *tokens* baseados na arquitetura *Transformer* [16], a evolução recente na aplicação dessas tecnologias migrou de chamadas diretas e lineares para o uso de estruturas de orquestração mais robustas. Um detalhamento teórico e histórico aprofundado sobre o funcionamento desses modelos de linguagem pode ser encontrado no levantamento conduzido por Zhao et al. [17].

Na prática, a organização dessas interações é viabilizada por meio de um *harness*, termo que se refere à estrutura lógica e às regras de orquestração que envolvem um modelo de linguagem para guiar seu processo de inferência, funcionando como uma camada de coordenação entre o *prompt* inicial do usuário e a saída do modelo [12], [18]. Um *harness* pode incluir etapas de planejamento, decomposição de tarefas, validação intermediária e composição de respostas finais. Variações na arquitetura do *harness* podem causar impactos significativos no desempenho do sistema como um todo, mesmo quando o modelo base permanece o

mesmo.

Esse conceito está diretamente relacionado à noção de *Sistemas Compostos de IA*, definida por Zaharia et al. [18] como arquiteturas que resolvem tarefas de inteligência artificial por meio de múltiplos componentes que interagem entre si, incluindo chamadas a modelos, recuperadores de informação e ferramentas externas. Segundo essa perspectiva, os ganhos mais significativos em aplicações de IA contemporâneas passaram a depender da engenharia de sistemas que combinam componentes especializados em fluxos coordenados, em vez de se apoiarem unicamente no aumento de escala dos modelos base [18].

Essa tendência se manifesta em ferramentas industriais que utilizam LLMs integrados a *harnesses* especializados para tarefas de engenharia e automação. Exemplos incluem o Claude Code [19], interface de linha de comando voltada ao desenvolvimento assistido por IA; o GPT Codex [20], otimizado para geração e compreensão de código; e o OpenCLI [21], voltado à automação de comandos de terminal. Essas ferramentas demonstram como a combinação de planejamento, decomposição de tarefas e validação iterativa expande a capacidade computacional útil dos modelos base.

B. Estratégias de Raciocínio e Refinamento

A literatura demonstra que a qualidade da inferência de um LLM pode ser melhorada de forma significativa quando o processo é estruturado em fases [2], [3]. O método *Chain-of-Thought* [2] e sua generalização *Tree of Thoughts* [3] mostraram a importância de induzir o modelo a explicitar passos intermediários de raciocínio. Abordagens como *ReAct* [5] ampliaram esse princípio ao combinar o raciocínio interno do modelo com a capacidade de invocar ferramentas externas, permitindo corrigir alucinações por meio de observações diretas do ambiente.

De forma complementar, pipelines baseados em ciclos de *feedback* iterativo também apresentam ganhos consistentes. O framework *Reflexion* [6] introduziu ciclos nos quais o modelo atua como seu próprio crítico, revisando saídas com base em reforço verbal externo. A abordagem de *Self-Debugging* [7] demonstrou que LLMs podem identificar e corrigir seus próprios erros quando guiados de forma sistemática. Na mesma direção, o método *Self-Refine* [8] consolidou a eficácia do refinamento iterativo ao demonstrar que modelos podem gerar *feedback* crítico sobre suas próprias respostas e utilizá-lo para melhorar as saídas de forma sucessiva, contornando limitações da geração em uma única etapa. Essa capacidade de autocritica e revisão contínua constitui uma premissa fundamental para a proposta deste trabalho.

Além dessas abordagens, métodos focados na decomposição de tarefas complexas também contribuem para a redução de erros lógicos e omissões. Técnicas de *prompting* baseadas em decomposição, como *Least-to-Most* [4], *Decomposed Prompting* [22] e *Plan-and-Solve* [23], sustentam que separar o planejamento da execução e dividir problemas em subproblemas menores melhora o desempenho do modelo, especialmente quando combinado com mecanismos de validação intermediária.

C. Degradação de Contexto

Embora as técnicas apresentadas nas subseções anteriores tenham demonstrado eficácia individual, muitas delas dependem do acúmulo contínuo de estado em uma única janela de contexto, o que pode agravar problemas de degradação à medida que o histórico cresce. Liu et al. [9] demonstraram empiricamente que LLMs apresentam perda significativa de desempenho quando precisam recuperar informações relevantes posicionadas no meio de sequências longas, fenômeno descrito como “perdidos no meio” (*lost in the middle*).

Na prática, esse comportamento se manifesta de diversas formas: compressão implícita de informações, perda de relevância contextual, aumento de inconsistências e maior incidência de alucinações em execuções prolongadas [9], [17]. A literatura contemporânea frequentemente associa esse conjunto de efeitos ao conceito de *context rot*, referindo-se à degradação progressiva da qualidade das respostas conforme o contexto acumula ruído, redundâncias e informações que competem pela atenção do modelo [9].

D. O Ralph Wiggum Loop

Diante dos problemas de degradação de contexto, o *Ralph Wiggum Loop* [11] surgiu na comunidade de engenharia de software como um paradigma de orquestração que aborda diretamente essa limitação. Idealizado por Geoffrey Huntley, o paradigma parte de um conjunto de requisitos fornecidos como entrada e os decompõe em histórias de usuário, conceito consolidado na disciplina de engenharia de software. Cada história é tratada de forma atômica: o modelo implementa uma solução, executa testes e verifica se os critérios de aceite foram satisfeitos; caso contrário, o ciclo se repete até que a história seja concluída com êxito. Ao término de cada história, seus resultados não são carregados diretamente para a execução seguinte, evitando o acúmulo progressivo de estado que caracteriza arquiteturas tradicionais de agentes.

Um aspecto central dessa abordagem é o princípio de *Fresh Context* [11]: ao iniciar cada nova história, o contexto de execução é completamente reinicializado. Antes de qualquer instrução sobre a nova história, os aprendizados gerais extraídos das histórias anteriores são injetados como primeira entrada nesse novo contexto, de forma estruturada e explícita. Dessa maneira, o paradigma preserva o conhecimento relevante acumulado sem herdar o ruído, as redundâncias e as informações obsoletas que degradam progressivamente a qualidade das respostas em execuções longas, mitigando ativamente o fenômeno de *context rot* [10].

Em sua formulação original, o *Ralph Wiggum Loop* foi concebido especificamente para fluxos de engenharia de software, com etapas voltadas à definição de funcionalidades, requisitos, critérios de aceite e integração com sistemas de versionamento como *Git*. Ainda assim, seus princípios de execução *stateless* e validação iterativa apresentam relevância direta para os problemas discutidos na literatura de LLMs. A validação acadêmica dessa abordagem foi reforçada pelo estudo de McComb et al. [24], que avaliou

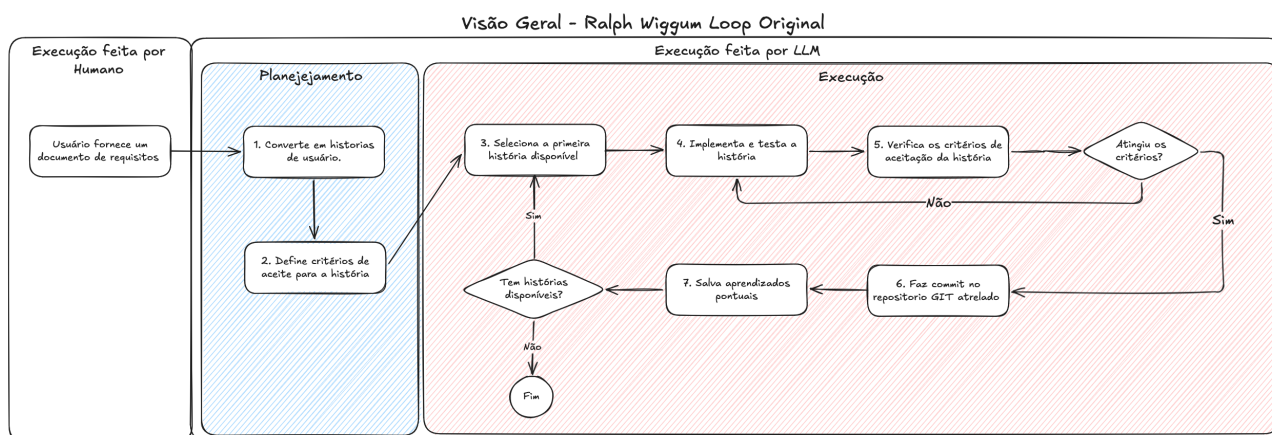


Figura 1. Ralph Wiggum Loop original.

o impacto de agentes baseados no *Ralph Wiggum Loop* na resolução de problemas complexos de design de engenharia, confirmando a eficácia do paradigma *stateless* em contextos que exigem raciocínio estruturado e iterativo. Este trabalho se apoia nesses fundamentos para justificar o uso do paradigma, adaptando sua orquestração para a resolução de questões de propósito geral em múltiplos domínios do conhecimento.

III. Metodologia

1. Análise macro do processo de thinking

O diagrama a seguir apresenta o fluxo macro do pipeline de *thinking* proposto neste trabalho, oferecendo uma visão consolidada de todas as fases que o compõem.

O pipeline inicia-se a partir de um *prompt* de entrada fornecido pelo usuário e segue etapas organizadas em loops aninhados: (i) definição dos critérios de aceitação e do formato de saída; (ii) decomposição do problema em tarefas atômicas independentes; (iii) processamento das tarefas no **Micro Loop**, onde cada sub tarefa é resolvida individualmente até sua conclusão; (iv) compilação dos resultados para avaliação no **Macro Loop** em relação aos critérios globais. Se a avaliação identificar lacunas, o pipeline cria tarefas complementares e reinicia o Micro Loop até que todos os critérios sejam satisfeitos. Esse encadeamento estruturado favorece convergência iterativa e consistência lógica na geração da resposta final.

Um aspecto central da arquitetura é a separação deliberada entre as fases de execução: cada etapa opera de forma *stateless* em relação às demais, sem carregar diretamente o histórico de raciocínio das fases anteriores. Em vez disso, o sistema preserva apenas artefatos estruturados, como os arquivos `planning.json` e `tasks_results.json`, que atuam como memória persistente e controlada entre as etapas. Essa escolha visa mitigar o contexto degradado (*context rot*) e garantir que cada chamada ao LLM opere com o menor contexto necessário, reduzindo o risco de contaminação por raciocínios prévios.

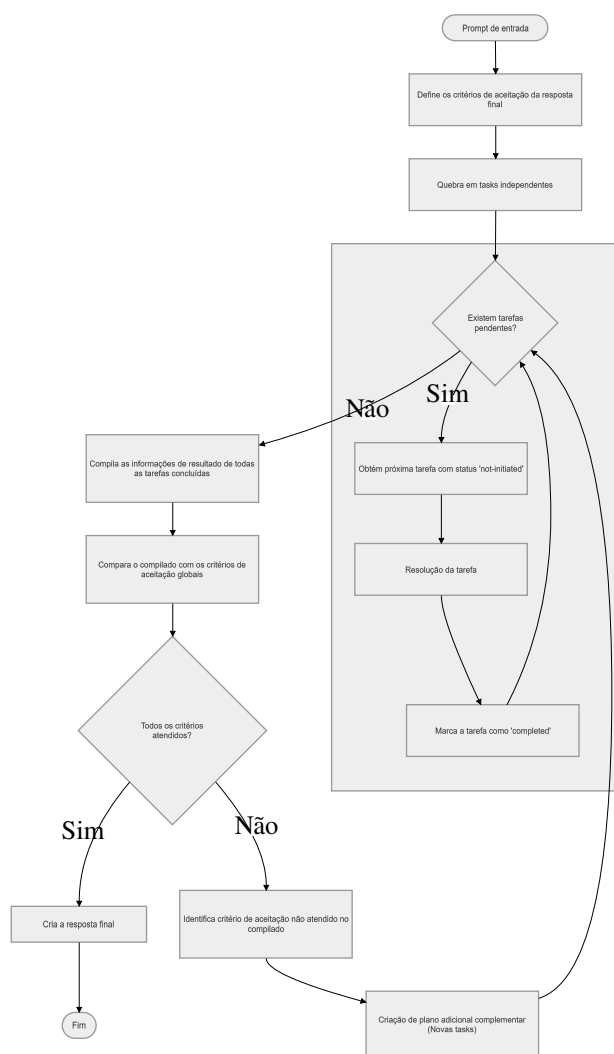


Figura 2. Fluxo macro do pipeline de *thinking*.

2. Análise micro das fases

A seguir, cada fase do pipeline é detalhada individualmente. Para cada fase, são descritos: os agentes de LLM envolvidos, os dados de entrada e saída, e o fluxo de controle interno. Essa granularidade tem o objetivo de tornar o processo inteiramente replicável por pesquisadores externos.

2.1 Agentes de LLM

Neste trabalho, adota-se uma definição restrita de *agente*: trata-se de um componente composto exclusivamente por um *prompt* pré-definido e uma chamada ao modelo de linguagem (LLM_CALL). Diferentemente de arquiteturas que atribuem a agentes acesso a ferramentas externas, chamadas a APIs, servidores MCP ou capacidades de navegação, os agentes aqui descritos operam de forma isolada, recebendo um contexto de entrada estruturado, processando via LLM e retornando uma saída textual ou estruturada em JSON. Essa restrição é intencional: ao eliminar dependências externas, torna-se possível avaliar de forma mais direta o impacto da orquestração e do *prompting* sobre a qualidade das respostas, isolando as variáveis de interesse do estudo.

Todos os *prompts* de agentes são redigidos em inglês. Estudos como os de Huang et al. [huang2024beyond](#) e Shi et al. [25] mostram que modelos de linguagem multilíngues tendem a produzir resultados de maior qualidade quando instruídos no idioma em que foram mais amplamente treinados, predominantemente o inglês, independentemente do idioma da questão de entrada. O trabalho de Qin et al. [qin2023crosslingual](#) reforça esse achado ao demonstrar que a combinação de *prompting* em inglês com raciocínio *zero-shot* em cadeia produz ganhos consistentes mesmo em línguas de menor representação nos dados de treinamento. A adoção do inglês como idioma padrão dos *prompts* de agentes visa, portanto, maximizar a qualidade do raciocínio interno do modelo, sem prejuízo ao idioma da resposta entregue ao usuário, que é determinado separadamente, conforme descrito na seção seguinte.

Adicionalmente, todos os *prompts* são construídos utilizando a sintaxe *Markdown*, com delimitadores explícitos de seção para separar instruções gerais, contexto de entrada e especificações de saída. Estudos como os de He et al. [he2024promptformatting](#) e Liu et al. [26] mostram que a formatação do *prompt* afeta de forma mensurável o desempenho dos modelos, e que otimizações conjuntas de conteúdo e formato produzem ganhos adicionais sobre a otimização exclusiva do conteúdo textual. A adoção de *Markdown* visa, portanto, tornar delimitadores, hierarquias e restrições mais salientes durante a inferência, contribuindo para respostas mais consistentes com as instruções fornecidas.

2.2 Definição dos critérios de aceite da resposta final

Esta fase é responsável por transformar o *prompt* bruto do usuário em dois artefatos que guiarão todo o processo posterior: (i) uma lista de critérios de aceitação que a

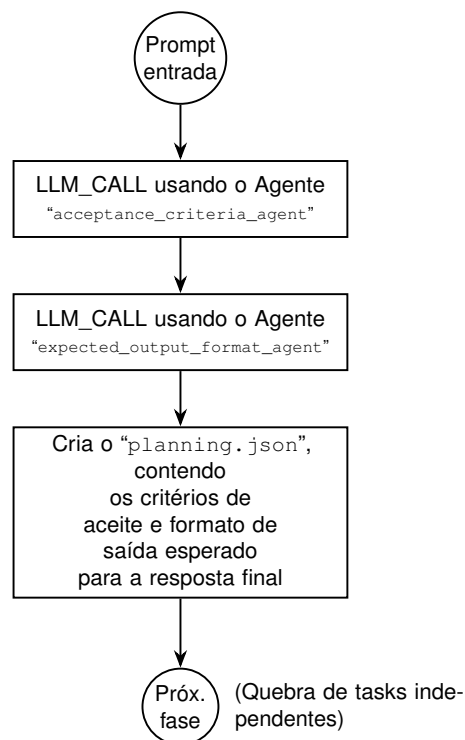


Figura 3. Fase de definição dos critérios de aceite da resposta final.

resposta final deverá satisfazer, e (ii) uma especificação do formato de saída esperado. Para isso, dois agentes são acionados em sequência a partir da mesma entrada.

O *acceptance_criteria_agent* recebe o *prompt* de entrada do usuário e é responsável por identificar e listar, de forma explícita, os critérios objetivos e verificáveis que uma resposta satisfatória deve cumprir. Em seguida, o *expected_output_format_agent* analisa o mesmo *prompt*, buscando por indicações explícitas de formato de saída, como “retorne somente a alternativa correta”, “responda em JSON” ou “elabore uma justificativa”. Caso nenhuma indicação seja encontrada, o agente infere o formato mais adequado ao tipo de tarefa: para questões de múltipla escolha, o padrão é “retornar somente a letra da alternativa correta, sem explicações adicionais”; para questões abertas, “texto corrido em formato Markdown”. O idioma da resposta final é tratado como parte integrante do formato de saída: se o usuário o especificou explicitamente, essa instrução é acatada; caso contrário, o agente infere o idioma a partir do idioma utilizado na entrada e o registra como especificação. Quando exemplos de saída são fornecidos pelo usuário, estes são incorporados ao campo correspondente.

Os resultados dessas duas chamadas são então consolidados em um arquivo JSON denominado *planning.json*, que servirá de artefato central de memória estruturada para todas as fases subsequentes. A estrutura do *planning.json* é inspirada no *prd.json* do Ralph Wiggum Loop original, porém estendida para incorporar as informações de critérios de aceitação e formato de saída.

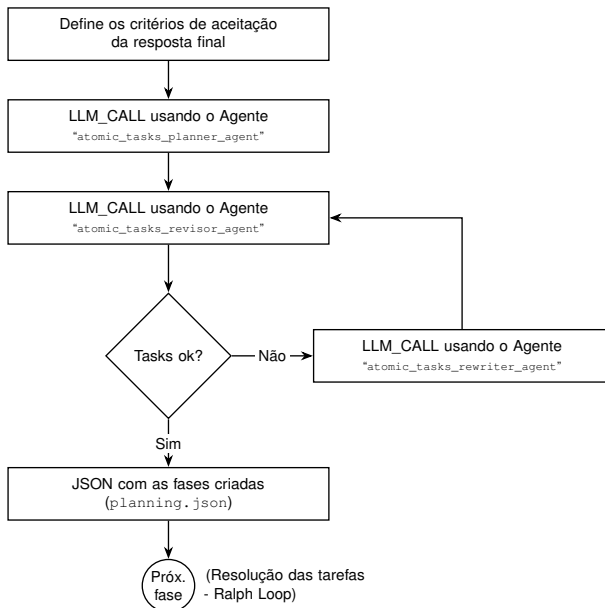


Figura 4. Fase de quebra de tarefas independentes.

```

{
  "general_infos": {
    "acceptance_criteria": [],
    "expected_output_format": {
      "format": "",
      "response_language": ""
    }
  },
  "tasks": []
}

```

O campo `tasks` é inicialmente vazio e será preenchido pela fase seguinte. A separação entre `general_infos` e `tasks` é proposital: as informações gerais são invariantes ao longo de todo o pipeline, enquanto as tarefas podem ser adicionadas iterativamente por planos complementares.

2.3 Quebra de Tasks Independentes

Esta fase é, em termos de responsabilidade lógica, a mais crítica de todo o pipeline: é aqui que se implementa o princípio central do Ralph Wiggum Loop, a decomposição do problema em unidades mínimas e independentes de execução. O agente `atomic_tasks_planner_agent` recebe como entrada o *prompt* do usuário e o campo `general_infos` do `planning.json` gerado na fase anterior, e a partir disso constrói um plano de tarefas atômicas.

O conceito de atomicidade empregado é preciso: uma tarefa é considerada atômica quando representa a menor unidade de trabalho possível e pode ser executada e avaliada de forma completamente independente das demais tarefas do plano, sem necessidade de acessar ou reutilizar os resultados de outras tarefas durante sua própria execução. Essa propriedade é fundamental para garantir o funcionamento correto do mecanismo de *fresh context*, pois, se tarefas dependessem umas das outras durante a execução, seria necessário carregar contexto acumulado, o que reintrodu-

ziria os problemas de degradação que a arquitetura busca mitigar.

Após a geração do plano inicial, o agente `atomic_tasks_revisor_agent` é acionado para verificar se todas as tarefas propostas satisfazem o critério de independência, retornando um JSON com um campo booleano de aprovação e uma justificativa. Se problemas forem identificados, o agente `atomic_tasks_rewriter_agent`, que incorpora em seu *prompt* todas as regras e restrições do planejador, é acionado para reescrever o plano com base na justificativa fornecida. Esse ciclo de revisão repete-se até que o revisor valide o plano, garantindo que apenas tarefas verdadeiramente independentes avancem para a fase de execução.

Cada tarefa do plano é representada no `planning.json` no seguinte formato:

```

{
  "id": "",
  "title": "",
  "description": "",
  "acceptance_criteria": [],
  "pass_phase": false
}

```

O campo `pass_phase` é inicializado como `false` para todas as tarefas e atualizado para `true` ao final da execução bem-sucedida de cada uma, funcionando como marcador de progresso para o mecanismo iterativo.

2.4 Resolução das tarefas — Ralph Loop

Esta fase implementa o núcleo iterativo do Ralph Wiggum Loop adaptado ao contexto deste trabalho. O processo inicia pela verificação do `planning.json`: enquanto existirem tarefas com `pass_phase: false`, o pipeline seleciona a próxima tarefa pendente e inicia um ciclo de tentativa, avaliação e revisão.

Cada iteração do ciclo consiste em: (i) uma chamada ao LLM contendo exclusivamente o conteúdo da tarefa atual, sem histórico acumulado de outras tarefas, seguida de (ii) uma chamada ao agente `task_outcome_reviewer_agent`, que compara a resposta gerada com os `acceptance_criteria` da tarefa. O revisor retorna um JSON com um campo booleano de aprovação e uma justificativa. Se o revisor aprovar a resposta, ela é salva no arquivo `tasks_results.json` e o campo `pass_phase` da tarefa é atualizado para `true`. Se reprovar, a justificativa é incorporada à próxima chamada ao LLM, configurando um ciclo de refinamento guiado, análogo aos mecanismos de *Reflexion* [6] e *Self-Debugging* [7].

No que diz respeito à persistência dos resultados, adota-se uma adaptação do princípio original do Ralph Wiggum Loop, que estabelece que “All memory lives in files and git, not the model”. Como o presente trabalho aplica o pipeline a cenários distintos do desenvolvimento de software, o mecanismo de *commit* em repositório Git é substituído pelo arquivo `tasks_results.json`, que acumula exclusivamente as respostas finais aprovadas de

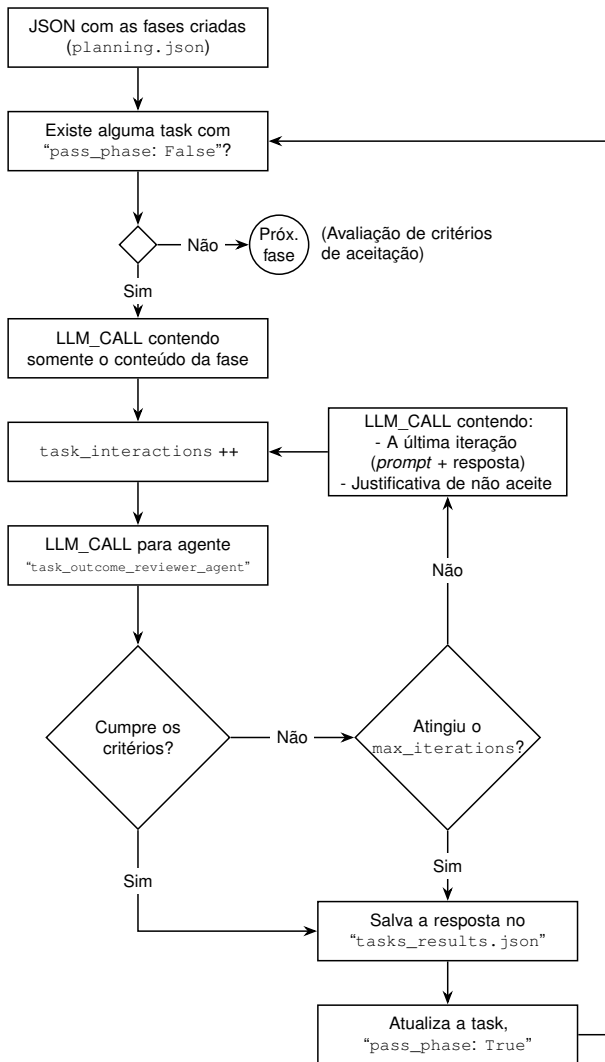


Figura 5. Fase de resolução iterativa das tarefas (Ralph Loop).

cada tarefa, sem registrar histórico de raciocínio, tentativas anteriores ou qualquer metadado do ciclo iterativo.

Para mitigar o risco de *deadlock*, implementa-se um mecanismo de controle denominado *pânico*. Um contador `task_interactions` é incrementado a cada chamada ao LLM dentro do ciclo de uma tarefa. Quando esse contador atinge o valor de `max_iterations`, a fase encerra automaticamente e a resposta disponível no momento é salva no `tasks_results.json`, mesmo que os critérios não tenham sido plenamente atendidos.

2.5 Avaliação dos critérios de aceitação

Ao término da resolução de todas as tarefas do plano, o pipeline consolida os resultados armazenados no `tasks_results.json` e os submete a uma avaliação global frente aos critérios de aceitação definidos em `general_infos.acceptance_criteria` do `planning.json`. Essa fase é executada pelo agente `final_response_acceptance_criteria_reviewer_agent`, que recebe ambos os artefatos como entrada.

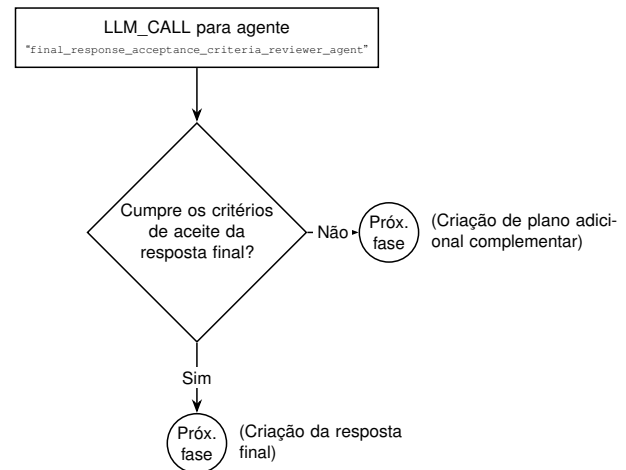


Figura 6. Fluxograma de avaliação dos critérios de aceitação da resposta final.

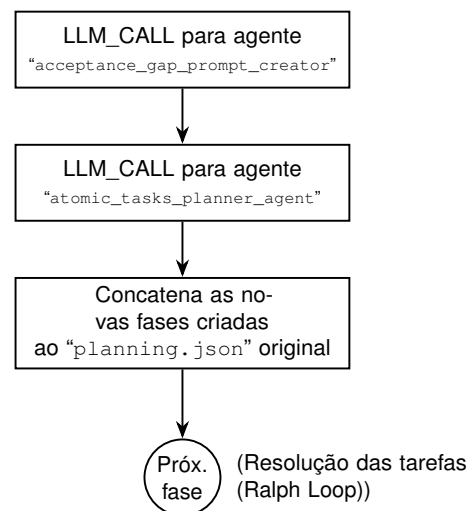


Figura 7. Criação de plano complementar para critérios não atendidos.

A lógica é análoga à dos revisores de tarefa individual, porém operando em um nível de abstração superior: em vez de avaliar uma resposta pontual contra critérios locais de uma tarefa, o agente avalia se o conjunto total de resultados gerados fornece insumos suficientes para construir uma resposta final que atenda a todos os critérios globais. Dois fluxos de saída são possíveis a partir dessa avaliação: caso todos os critérios sejam considerados atendidos, o pipeline avança para a criação da resposta final; caso contrário, avança para a criação de um plano adicional complementar.

2.6 Criação de plano complementar

Quando a avaliação global indica que os critérios de aceitação não foram plenamente atendidos, o pipeline não retorna ao início, o que implicaria descartar e reprocessar etapas já concluídas. Em vez disso, adota-se uma estratégia de afunilamento progressivo: o agente `acceptance_gap_prompt_creator` analisa a justificativa retornada pelo revisor global e produz um novo

prompt no estilo de entrada de usuário, descrevendo exclusivamente o que ainda precisa ser resolvido para cobrir as lacunas identificadas.

Esse *prompt* é então submetido ao `atomic_tasks_planner_agent`, o mesmo agente da fase de quebra de tarefas independentes, que gera um novo conjunto de tarefas complementares. Essas novas tarefas são concatenadas ao campo `tasks` do `planning.json` original, e o pipeline retorna à fase de resolução das tarefas para executá-las.

Esse mecanismo constitui, na prática, um segundo nível de loop que garante a convergência iterativa do pipeline: a cada ciclo, o conteúdo gerado aproxima-se do que é necessário para satisfazer todos os critérios de aceitação, sem desperdiçar o trabalho já realizado nas iterações anteriores.

2.7 Criação da resposta final

A fase de criação da resposta final assume que o `tasks_results.json` contém todos os insumos necessários para compor uma resposta que satisfaça integralmente os critérios de aceitação globais, premissa garantida pela fase de avaliação anterior. É nesta fase, e somente nesta, que o conteúdo do `tasks_results.json` é efetivamente recuperado e utilizado.

O agente `final_response_composer_agent` recebe como entrada o arquivo completo de resultados, o campo `general_infos.expected_output_format` do `planning.json` e o *prompt* original do usuário, utilizando-os em conjunto para compor a resposta final no formato e idioma especificados. O retorno desse agente é a saída direta do pipeline ao usuário, encerrando o processo de *thinking*.

IV. Cronograma do projeto

V. Referências

Referências

- [1] Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab e C. Finn, “Meta-Harness: End-to-End Optimization of Model Harnesses,” *arXiv preprint arXiv:2603.28052*, 2026.
- [2] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *arXiv preprint arXiv:2201.11903*, 2022.
- [3] S. Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” *arXiv preprint arXiv:2305.10601*, 2023.
- [4] D. Zhou et al., “Least-to-most prompting enables complex reasoning in large language models,” *arXiv preprint arXiv:2205.10625*, 2022.
- [5] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [6] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan e S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” *arXiv preprint arXiv:2303.11366*, 2023.
- [7] X. Chen, M. Lin, N. Schärli e D. Zhou, “Teaching Large Language Models to Self-Debug,” *arXiv preprint arXiv:2304.05128*, 2023.
- [8] A. Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” *arXiv preprint arXiv:2303.17651*, 2023.
- [9] N. F. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, v. 12, pp. 277–294, 2024.
- [10] K. Hong, A. Troynikov e J. Huber, “Context Rot: How Increasing Input Tokens Impacts LLM Performance,” Chroma, rel. técn., jul. de 2025. endereço: <https://trychroma.com/research/context-rot>
- [11] G. Ghuntley, *Ralph Loop: A Structured Orchestration Pattern for LLM Inference*, Online. Disponível em: <https://ghuntley.com/specs/ralph/>, Acesso em: abr. 2026, 2025.
- [12] Z. Wang et al., “Beyond the Strongest LLM: Multi-Turn Multi-Agent Orchestration vs. Single LLMs on Benchmarks,” *arXiv preprint arXiv:2509.23537*, 2025.
- [13] D. Hendrycks et al., “Measuring Massive Multitask Language Understanding,” em *International Conference on Learning Representations*, 2021.
- [14] D. Rein et al., “GPQA: A Graduate-Level Google-Proof Q&A Benchmark,” *arXiv preprint arXiv:2311.12022*, 2023.
- [15] T. B. Brown et al., “Language Models are Few-Shot Learners,” em *Advances in Neural Information Processing Systems*, 2020.
- [16] A. Vaswani et al., “Attention is All You Need,” em *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [17] W. X. Zhao et al., “A Survey of Large Language Models,” *arXiv preprint arXiv:2303.18223*, 2023.
- [18] M. Zaharia et al., *The Shift from Models to Compound AI Systems*, Berkeley Artificial Intelligence Research Blog. Disponível em: <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>, Acesso em: mai. 2026, 2024.
- [19] Anthropic, *Claude Code*, <https://www.anthropic.com/claude-code>, 2026.
- [20] M. Chen et al., “Evaluating Large Language Models Trained on Code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [21] *OpenCLI*, <https://opencli.ai/>, 2026.

Tabela I. Cronograma das atividades do projeto.

Atividade	fev	mar	abr	mai	jun	jul	ago	set	out	nov
Revisão bibliográfica e refinamento da proposta	X	X	X	X						
Definição da arquitetura		X	X	X	X					
Desenvolvimento do Paper		X	X	X	X	X	X	X	X	X
Implementação prática do código				X	X	X	X	X	X	
Análise final									X	X

- [22] T. Khot et al., “Decomposed prompting: A modular approach for solving complex tasks,” *arXiv preprint arXiv:2210.02406*, 2022.
- [23] L. Wang et al., “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” *arXiv preprint arXiv:2305.04091*, 2023.
- [24] C. McComb et al., “Supervising Ralph Wiggum: Exploring a Metacognitive Co-Regulation Agentic AI Loop for Engineering Design,” *arXiv preprint arXiv:2603.24768*, 2026.
- [25] F. Shi, M. Suzgun, M. Freitag, X. Wang, M. Surdeanu e J. Wei, “Language Models are Multilingual Chain-of-Thought Reasoners,” em *International Conference on Learning Representations*, 2023.
- [26] Y. Liu et al., “Beyond Prompt Content: Enhancing LLM Performance via Content-Format Integrated Prompt Optimization,” *arXiv preprint arXiv:2502.04295*, 2025.